The end-to-end project

California housing

LECTURE 2 | GÉRON CH. 2

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Last lecture we looked at the data and split it. Everything you need from that, in one table:

What we found	Why it matters today
One categorical column, `ocean_proximity`, one level with n=5	it has to be encoded, and that level will be absent from some folds
207 missing values in `total_bedrooms`	something must fill them, and that something is *learned* from data
Features on wildly different scales	they have to be scaled, and scaling is also learned from data
A target capped at $500,000	4.8% of rows carry a label that is not the answer
16,512 training / 4,128 test, stratified on income	the test rows are sealed until the last ten minutes of today

✗ Three of those five say *learned from data*. That is the whole subject of the middle of this lecture.

Today

1. **The mathematics** — least squares and the normal equation (20 min)
2. **Measuring generalisation** — what a score computed on the wrong rows tells you (15 min)
3. **The method** — pipelines, cross-validation, tuning (40 min)
4. **What the number means** — the test set, once (15 min)

By the end you will have a defensible number for this problem, and — more usefully — the procedure that makes any number defensible.

THE MATHEMATICS

Least squares and the normal equation

Géron §4.1, brought forward — today we fit a linear model, so today we should know what fitting one

20 minutes · examinable

The question

You called `LinearRegression().fit()`.

What did it actually compute — and when does that computation *fail*?

> **WHY YOU NEED THE ANSWER**
>
> The failure condition is exactly the warning from the previous lecture about engineering features as weighted sums.

The linear model

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- $\hat{y}$ — the predicted value
- n — the number of features
- x_i — the i-th feature value
- θ_j — the j-th model parameter, including the bias term θ_0

A weighted sum of the input features, plus a constant called the *bias* or *intercept* term.

Vectorised form

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

where $\mathbf{x}$ contains x_0 through x_n with $x_0 \equiv 1$, so the dot product reproduces the bias term.

> In machine learning, vectors are usually **column vectors**. With that convention the prediction is $\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$ — the same quantity, written as a single-cell matrix rather than a scalar.

What “fit” means

Training the model means setting $\boldsymbol{\theta}$ so the model best fits the training set. To do that we need a measure of how badly it fits.

$$\mathrm{MSE}(\mathbf{X}, y, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

MEAN SQUARED ERROR OF A LINEAR HYPOTHESIS

Why minimise MSE and not RMSE?

Because $\sqrt{\cdot}$ is strictly increasing on $[0, \infty)$:

$$\arg\min_{\boldsymbol{\theta}} \sqrt{f(\boldsymbol{\theta})} \;=\; \arg\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$$

Same minimiser, simpler algebra.

> *Learning algorithms often optimise a different function during training than the one used to evaluate the final model — because it is easier to optimise, or because it carries extra terms needed only during training.*

In matrix form

Stack the instances as rows of $\mathbf{X}$ and the labels into y:

$$\mathrm{MSE}(\boldsymbol{\theta}) = \frac{1}{m} \|\mathbf{X}\boldsymbol{\theta} - y\|_2^2$$

We are minimising a squared Euclidean norm. **That is a geometric statement**, and it is the whole content of this thread.

Differentiate and set to zero

$$\nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta}) \;=\; \frac{2}{m}\,\mathbf{X}^{\intercal}(\mathbf{X}\boldsymbol{\theta} - y)$$

Setting the gradient to zero at the minimiser $\hat{\boldsymbol{\theta}}$:

$$\mathbf{X}^{\intercal}\left(\mathbf{X}\hat{\boldsymbol{\theta}} - y\right) \;=\; \mathbf{0}$$

A stationary point. Is it a minimum?

Why that stationary point is *the* minimum

The Hessian of the MSE is

$$\nabla^2_{\boldsymbol{\theta}} \mathrm{MSE} = \frac{2}{m} \mathbf{X}^\intercal \mathbf{X}$$

For any vector $\mathbf{v}$, $\mathbf{v}^\intercal \mathbf{X}^\intercal \mathbf{X} \mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0$, so the Hessian is positive semi-definite and the MSE is **convex**.

✓ A convex function has no local minima — only a global one. Any stationary point is it.

This is also why gradient descent, in Lecture 4, is guaranteed to find the same answer.

The normal equation

Rearranging $\mathbf{X}^\intercal \mathbf{X} \hat{\boldsymbol{\theta}} = \mathbf{X}^\intercal y$:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X})^{-1} \mathbf{X}^\intercal y$$

A CLOSED-FORM SOLUTION — NO ITERATION REQUIRED

A closed-form equation is composed only of a finite number of constants, variables and standard operations. No infinite sums, no limits, no integrals.

Least squares *is* orthogonal projection

Read the stationarity condition again: every column of $\mathbf{X}$ is orthogonal to the residual.

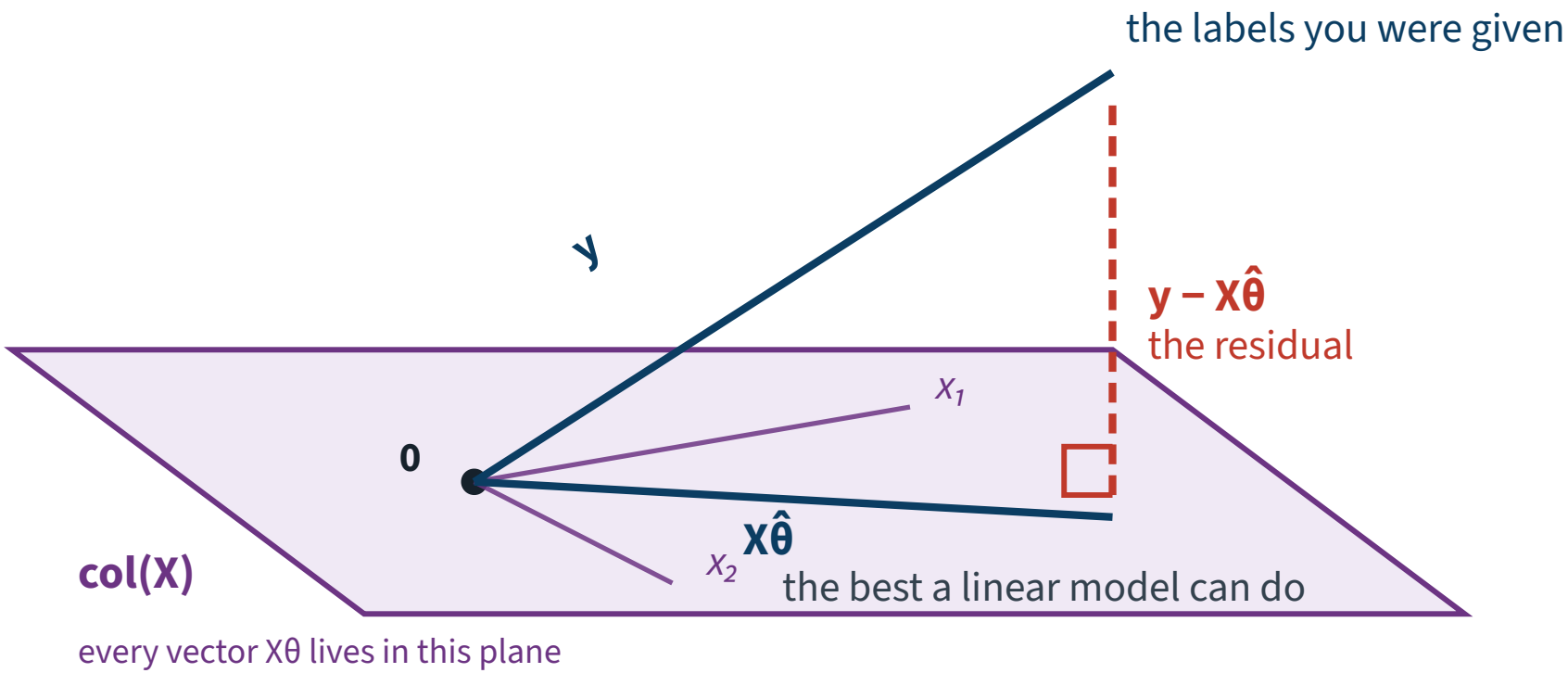
$$\mathbf{X}^\top \underbrace{\left(\mathbf{X}\hat{\boldsymbol{\theta}} - y\right)}_{\text{residual } \mathbf{r}} = \mathbf{0}$$

- $\mathbf{X}\boldsymbol{\theta}$ ranges over the **column space** of $\mathbf{X}$ as $\boldsymbol{\theta}$ varies
- Minimising $\|\mathbf{X}\boldsymbol{\theta} - y\|_2$ finds the point of that subspace closest to y
- The closest point of a subspace to a vector is its **orthogonal projection**

> Fitting a linear model is projecting the label vector onto the span of the features. The residual is what the features cannot reach.

The whole thread, in one picture

$$X^\top \left(X\hat{\boldsymbol{\theta}} - \mathbf{y} \right) = \mathbf{0}$$



Compute it on our own data

```
import numpy as np
from sklearn.preprocessing import add_dummy_feature

housing_num = X_train.select_dtypes(include=[np.number])   # 16,512 x 8
X = make_pipeline(SimpleImputer(strategy="median"),
                  StandardScaler()).fit_transform(housing_num)
y = y_train.values

X_b = add_dummy_feature(X)                # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

The `@` operator is matrix multiplication. PyTorch, TensorFlow and JAX all support it; pure Python lists do not. Eight features, so $\hat{\theta}$ has nine entries.

And compare with the library

```
>>> theta_best[:3].round(1)
array([206333.5, -85371.4, -90829.7])
>>> lin_reg = LinearRegression().fit(X, y)
>>> np.abs(theta_best - np.r_[lin_reg.intercept_, lin_reg.coef_]).max()
6.4e-10
```

Identical to floating-point precision. Scikit-Learn separates the bias term (`intercept_`) from the feature weights (`coef_`); we stacked them back together to compare.

> **LOOK AT Θ_0**
>
> $206,334 — and the mean price of the training set is $206,334. With centred features the intercept *is* the mean of the target, so the constant predictor — which we measure later in this lecture — is exactly this model with all eight weights set to zero. Least squares spends those eight weights to get from $115,311 to $68,233.

Verify the geometry, on the same data

If the geometry is right, the residual must be orthogonal to every column of $\mathbf{X}$ — so $\mathbf{X}^\top \mathbf{r}$ should be numerically zero.

```
residual = X_b @ theta_best - y
print(np.abs(X_b.T @ residual).max())        # 7.1e-06
print(np.abs(X_b.T @ y).max())             # 3.4e+09 — the scale to judge it by
```

Not exactly zero: floating-point arithmetic. But 7.1×10^{-6} against a scale of 3.4×10^{9} is a relative 2×10^{-15} — machine epsilon, on a double.

✓ A cheap, decisive check that your solver did what the mathematics says it should. Note that “small” is meaningless until you divide by something.

When does this break?

The equation requires $\left(\mathbf{X}^\intercal \mathbf{X}\right)^{-1}$ to exist.

$\mathbf{X}^\intercal \mathbf{X}$ is $(n+1) \times (n+1)$. It is invertible if and only if it has full rank — equivalently, if and only if $\mathbf{X}$ has linearly independent columns.

✗ Two ways to lose that.

Failure 1 · More features than instances

If $m < n$, then $\mathrm{rank}(\mathbf{X}) \leq m < n + 1$, so $\mathbf{X}^\intercal \mathbf{X}$ is singular.

> Infinitely many parameter vectors fit the data exactly. The problem is not that there is no solution — it is that there are too many.

Common in genomics, text, and any wide-feature setting.

Failure 2 · Collinear features

If one column is a linear combination of others, the columns are dependent and $\mathbf{X}^\intercal\mathbf{X}$ is singular again.

> **THE PREVIOUS LECTURE'S WARNING, EXPLAINED**
>
> "When creating new combined features, avoid simple weighted sums of existing features."
>
> A weighted sum of existing columns *is* a linear combination. You would be manufacturing the singularity yourself.

Near-singular is also a problem

Exact collinearity is rare; *near* collinearity is common.

- $\mathbf{X}^\intercal \mathbf{X}$ is invertible but ill-conditioned
- Tiny changes in the training set produce large changes in $\hat{\boldsymbol{\theta}}$
- The model becomes **numerically unstable**

In Lecture 5 we will repair this with ridge regression, which adds $\alpha \mathbf{I}$ and makes the matrix invertible for every $\alpha > 0$.

What Scikit-Learn actually does

It does not invert anything. It computes

$$\hat{\boldsymbol{\theta}} = \mathbf{X}^{+} y$$

where $\mathbf{X}^{+}$ is the **pseudoinverse** — specifically the Moore–Penrose inverse.

```
np.linalg.pinv(X_b) @ y                    # same result
```

Computed by SVD

Singular value decomposition factors the training matrix as

$$\mathbf{X} = \mathbf{U}\,\boldsymbol{\Sigma}\,\mathbf{V}^{\mathsf{T}} \qquad \Longrightarrow \qquad \mathbf{X}^{+} = \mathbf{V}\,\boldsymbol{\Sigma}^{+}\mathbf{U}^{\mathsf{T}}$$

To build $\boldsymbol{\Sigma}^{+}$: take $\boldsymbol{\Sigma}$, set to zero every value below a tiny threshold, replace the remaining values by their reciprocals, and transpose.

✓ The pseudoinverse is always defined, even when the inverse is not.

What it costs

Method	In n (features)	In m (instances)
Normal equation	$O(n^{2.4})$ to $O(n^3)$	$O(m)$
SVD (Scikit-Learn)	$O(n^2)$	$O(m)$

- Double the features with the normal equation → roughly 5 to 8 times slower
- Double the features with SVD → roughly 4 times slower
- Both are **linear in the number of instances** — they handle large training sets well, provided the data fits in memory

✗ Both become very slow when n grows large — say 100,000 features. That is what gradient descent is for.

What that derivation buys you

- Fitting a linear model minimises $\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2$
- The minimiser satisfies $\mathbf{X}^\intercal(\mathbf{X}\hat{\boldsymbol{\theta}} - y) = \mathbf{0}$: the residual is orthogonal to every feature
- Hence $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal y$, when the inverse exists
- It fails to exist when $m < n$ or when features are collinear
- The SVD-based pseudoinverse is always defined and cheaper

> **THREE THINGS YOU CAN NOW DO**
>
> Say why `LinearRegression` never fails on this data even though the matrix could be singular; say why the collinearity warning at the end of the last lecture was not pedantry; and, in Lecture 5, see immediately why adding $\alpha\mathbf{I}$ repairs the invertibility for every $\alpha > 0$.

Derive the normal equation; state and justify the invertibility condition.

EXAMINABLE

BEFORE WE BUILD ANYTHING

Measuring generalisation

15 minutes · examinable

Start with a number that means nothing

Fit an unconstrained decision tree on the 16,512 training districts, then score it on those same 16,512 districts.

0.0

No error at all.

Two explanations. Only one of them is flattering.

Explanation 1 · The model is perfect

Housing prices would have to be an exact deterministic function of nine census attributes — eight numbers and a category.

- Two identical districts would always have identical median prices
- No measurement noise anywhere in a census
- No influence from anything not in our nine columns

✗ Reject.

Explanation 2 · The model memorised

A decision tree left unconstrained keeps splitting until every leaf is pure.

> With enough depth it can isolate **every single training district into its own leaf**. Then predicting the training set is a lookup, and a lookup has zero error.

This is **overfitting**: excellent on the training data, useless on anything else.

> *Such a model is called nonparametric: not because it has no parameters, but because the **number** of parameters is not fixed before training, so the structure is free to follow the data. That freedom is exactly what lets it memorise.*

What the zero actually tells you

Overfitting is a property of the *model*.

✗ A training score cannot *see* it.

> **THE CONDITION**
>
> Any score computed on the rows a model was fitted to measures how well it reproduces what it has already seen. For a model flexible enough to memorise, that is not evidence about anything else.

Linear regression on the same rows gives $68,233. That one happens to be honest to within $49 — but only because the model is too rigid to memorise, and nothing in the number tells you which case you are in.

“Then use the test set”

✗ No — not yet.

We are still choosing between models and tuning them. Every time a choice is made by looking at the test score, that score stops being an estimate of performance on unseen data and becomes a target we are fitting to.

> The test set is used **once**, at the very end, to estimate the generalisation error of a system we have already committed to.

This is the same argument as the specimen exam question in the last lecture, where a loop chose α by scoring on `X_test`. We answer the rest of that question at the end of today.

The same error, one step earlier

A score can also be corrupted before any model is fitted:

```
scaler = StandardScaler()
housing_scaled = scaler.fit_transform(housing_full)      # all 20,640 rows
train_set, test_set = train_test_split(housing_scaled)   # then split
```

DATA LEAKAGE — THE CONDITION

A quantity computed from rows outside the training set enters the training path. Here the mean and standard deviation were computed from a set that includes the test rows, so the model is evaluated on rows whose own values helped define the transformation applied to them.

It raises no exception and the reported score improves. How *much* it improves is not knowable from the code — sometimes nothing, sometimes everything — which is why the rule is procedural rather than a matter of judging each case.

Not knowable in advance. Measurable afterwards.

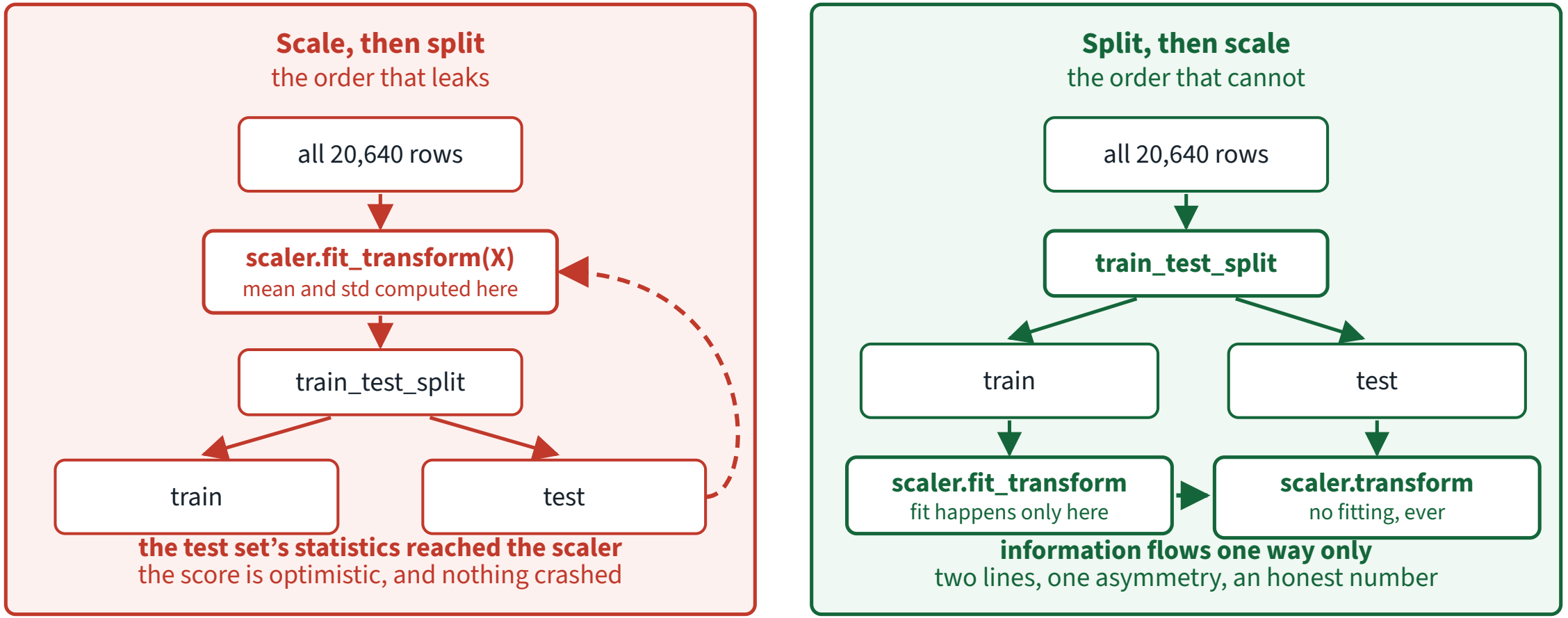
Same data, same models, twenty seeds — scaled before the split against scaled after it. All figures are dollars of RMSE.

Model	seed 42	mean difference, 20 seeds	sd
Linear regression	0.88	0.02	0.31
Random forest	57.17	3.89	47.87
PCA(4), then linear	15.90	1.85	16.34

✗ Against a split-to-split spread of 1,391 on the same RMSE, every one of those is nothing — and the sign flips from seed to seed. Report the seed-42 column on its own and you have published noise.

Which is the argument for the rule, not against it: the cost here was nothing, the cost elsewhere is unbounded, and you cannot tell which case you are in from the code.

The two orders



Same two operations, opposite order. In the upper flow the test rows help define the transformation applied to them; in the lower one information moves one way only.

The rule, stated once

> **NEVER VIOLATE THIS**
>
> Fit every transformer on the **training set only**. Never call `fit()` or `fit_transform()` on the validation set, the test set, or new data. Once fitted, you may `transform()` anything.

This applies to imputers, scalers, encoders, dimensionality reduction — everything that learns anything from data.

✗ We will not rely on remembering it. The rest of this lecture builds an object that makes violating it structurally impossible.

What we need, then

Requirement	Because
An estimate of error on rows the model has not seen	a training score cannot see overfitting
… without touching the test set	it is spent the first time it influences a choice
… with a *spread*, not one number	two estimates are only different if the spread says so
… and all preprocessing refitted inside it	otherwise the estimate leaks and is optimistic

All four are satisfied by one procedure.

THE METHOD

Cross-validation, pipelines and tuning

40 minutes · examinable

Today’s notebook

Open Lecture 2 in Google Colab

Everything from here to the end of the lecture, runnable. It repeats the Lecture 1 load and split in its first two cells, so it stands on its own.

Fix 1 · Hold out a validation set

Split the *training* set again: a smaller training set, and a **validation set** (also called the development set). Train the candidates on the reduced set, select on the validation set, retrain the winner on the full training set, and evaluate *once* on the test set.

Validation set too small

Evaluations are imprecise; you may select a suboptimal model by chance.

Validation set too large

The remaining training set is much smaller than the full one — like selecting a sprinter to run a marathon.

✓ Solution: many small validation sets, used in rotation.

Fix 2 · k-fold cross-validation

Split the training set into k non-overlapping **folds**. Train and evaluate k times, holding out a different fold each time.

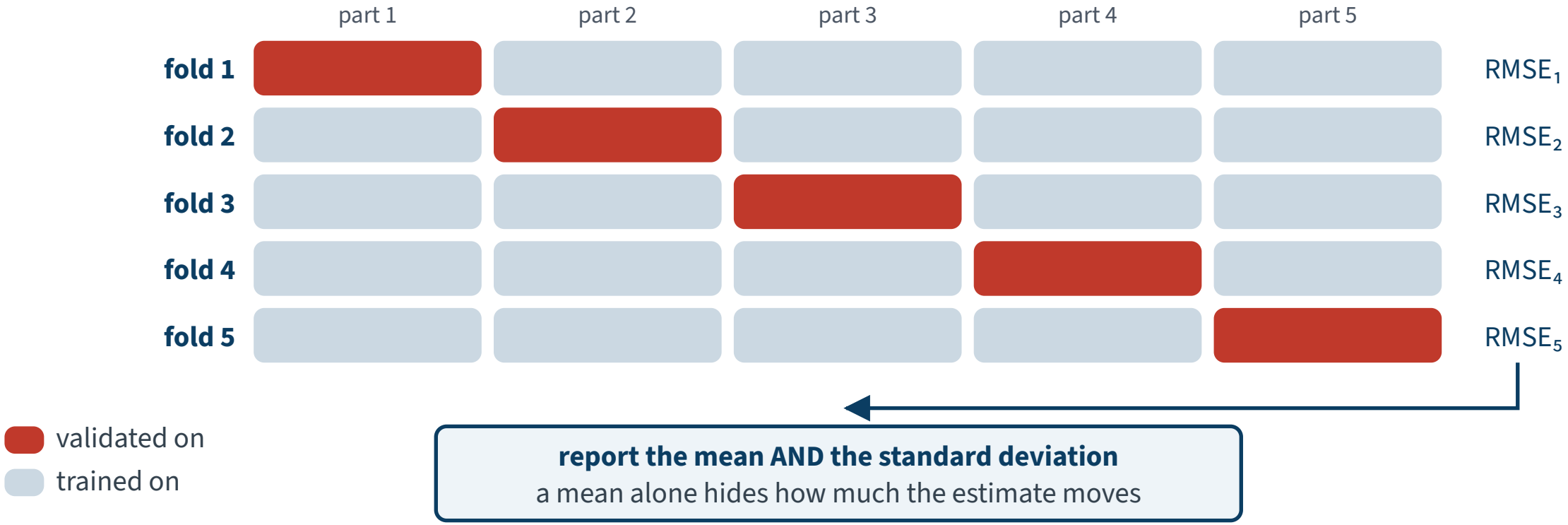
$$\text{score} = \frac{1}{k}\sum_{j=1}^{k}\text{RMSE}(\text{model trained without fold } j,\ \text{fold } j$$

Cost: you train the model k times.

Every row is a complete, independent fit

The training set, partitioned five ways

Every row is one complete fit. Every row scores on data that fit has never seen.



k = 5 drawn; the lecture uses k = 10

Five fits to get one number — and a second number, the spread, that a single holdout could never give you.

In code

```
from sklearn.model_selection import KFold, cross_val_score

# not cv=10: the default KFold does not shuffle
cv = KFold(n_splits=10, shuffle=True, random_state=42)

tree_rmses = -cross_val_score(tree_reg, X_train, y_train,
                              scoring="neg_root_mean_squared_error", cv=cv)
# about 25 s
```

Why the minus sign. Scikit-Learn's cross-validation expects a *utility* function (greater is better), not a cost function. The scorer is the negative RMSE, so flip the sign back.

Why the explicit `KFold`. `cv=10` takes ten contiguous blocks in row order. Two of you whose frames are sorted differently then get different folds and cannot see why your numbers disagree.

The runtime comment is there so that four minutes of no output is not read as "it hung" and interrupted. Timings are for a free Colab CPU runtime and are **not examinable**.

The tree, measured honestly

```
>>> pd.Series(tree_rmses).describe()
count       10.000000
mean     68573.733113
std       2014.711361
min      65499.052101
25%      66693.914880
50%      69070.930754
75%      69962.630213
max      70958.417942
```

The honest number for that same tree: $68,574, not zero.

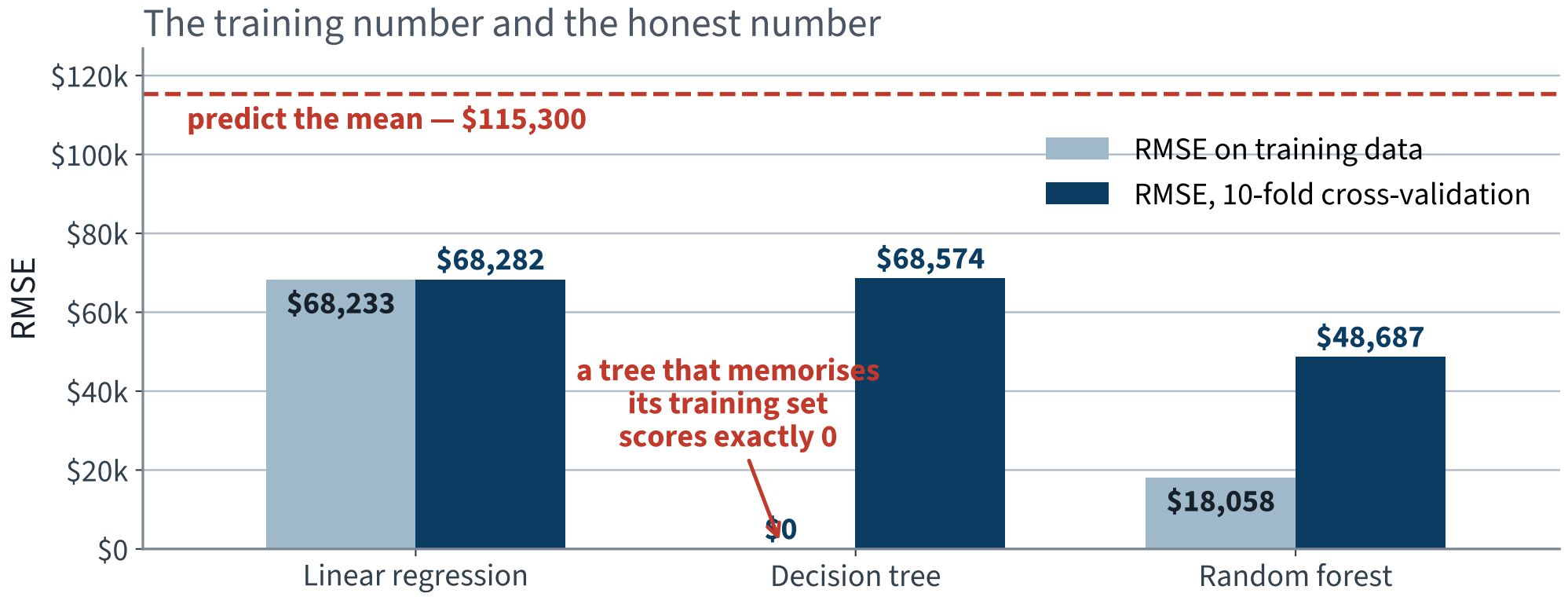
And something a single number cannot give us: a spread. The ten folds run from $65,499 to $70,958, so any comparison that turns on less than a couple of thousand dollars is not a comparison.

All three, measured honestly

Model	Training RMSE	Cross-validated RMSE
Predict a constant	$115,311	$115,300
Linear regression	$68,233	$68,282
Decision tree	✗ $0	$68,574
Random forest	$18,058	**$48,687 ✓**

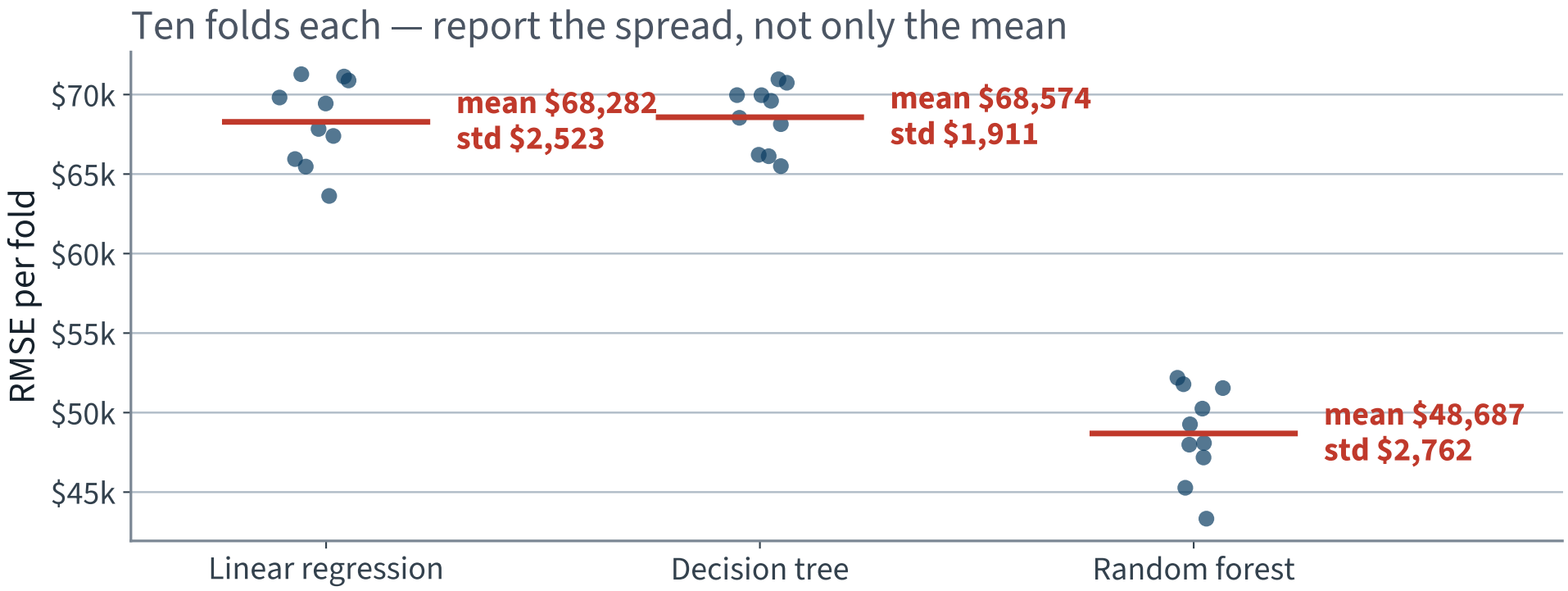
Now the table is readable. The gap between the two columns *is* the overfitting — and the first row, which cannot overfit anything, has no gap at all.

The gap between the columns *is* the overfitting



✗ The decision tree has no training bar at all. Its perfect score is a bar of height zero — and that is exactly how much it told you.

Ten folds, not one number



All three spreads are around ±$2,000–$3,000 — and that is mostly *which districts landed in the fold*, not the model. Any difference smaller than this is not a difference.

Reading the table

- **Linear regression** — the two columns nearly agree. It is not overfitting; it is **✗ underfitting**
- **Decision tree** — $0 against $68,574. Severe overfitting — and once measured honestly it is not better than the linear model it appeared to crush
- **Random forest** — $18,058 against $48,687. Still overfitting, but by far the best honest score, and 58% below the constant baseline

Fixes for overfitting: simplify the model, constrain it, or obtain more training data.

Is $68,574 worse than $68,282?

The tree's mean is $292 above the linear model's, and the folds scatter by $2,000. So compare them *fold by fold*: both models saw the same ten folds, and the shared fold-to-fold noise cancels in the difference.

Per-fold difference	Mean	Std	Same sign in all 10?
Decision tree − linear regression	+$292	$1,634	✗ no
Random forest − linear regression	−$19,594	$1,650	yes ✓

✓ The forest is genuinely better — it averages many trees fitted on random subsets, and if their errors are not strongly correlated the average is more accurate than any of them. (Lecture 7 makes that precise, and derives it.) The tree is indistinguishable from the linear model — a paired t-test gives $p = 0.59$, and on six of the ten folds the tree is the *better* of the two.

✗ Say "the tree is worse" and you have read a ranking out of noise. Two numbers are only different if you have shown that they are.

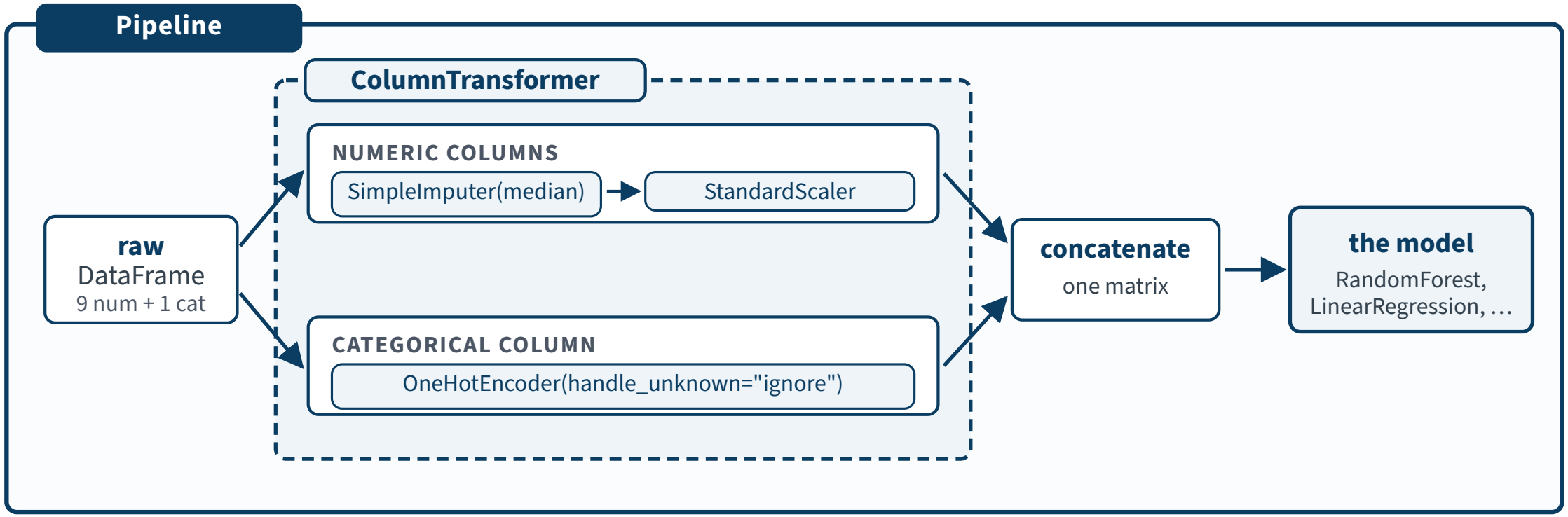
Fix 3 · Pipelines make leakage structurally impossible

```
full_pipeline = Pipeline([
    ("prep",  preprocessing),                    # the ColumnTransformer
    ("model", RandomForestRegressor(random_state=42)),
])
```

When `cross_val_score` holds out fold j, the entire pipeline — imputer, encoder, scaler, model — is fitted on the other folds only.

✓ The transformers never see the held-out fold. Leakage cannot occur by accident.

What that object actually is



One object, so `cross_val_score` refits *all* of it on each fold's training part — the imputer's median and the scaler's mean never see the validation fold.

Fix 4 · Tune, instead of guessing

```
from sklearn.model_selection import GridSearchCV

param_grid = {"model__max_features": [4, 6, 8, 10, 12],
              "model__n_estimators": [30, 100, 200]}

grid_search = GridSearchCV(full_pipeline, param_grid, cv=cv,
                           scoring="neg_root_mean_squared_error", n_jobs=-1)
grid_search.fit(X_train, y_train)      # 15 x 10 = 150 fits, about 4 minutes
```

`model__max_features` reads as: the step named `model`, its parameter `max_features`. Double underscores address any hyperparameter of any estimator, however deeply nested.

Four minutes with no output. That is the cell working, not the cell hanging — do not interrupt it. *(Timing not examinable.)*

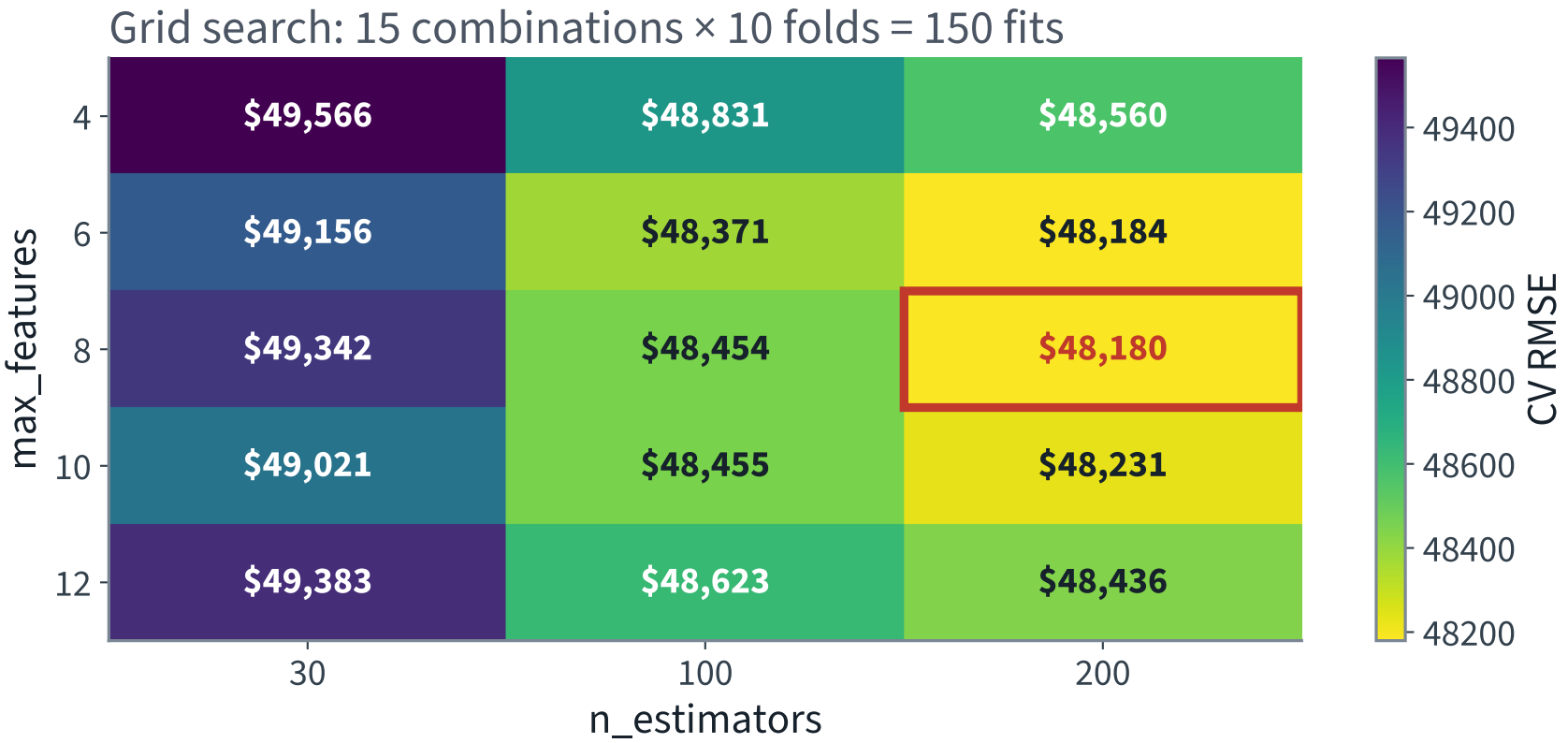
Grid search result

```
>>> grid_search.best_params_
{'model__max_features': 8, 'model__n_estimators': 200}
>>> -grid_search.best_score_
48179.64299597594
```

$48,180, improved from $48,687 — a gain of $508, against a fold spread of $2,762. Note that carefully: 150 fits bought us something we cannot cleanly distinguish from noise.

> **AND READ THE BOUNDARY**
>
> 200 was the *largest* value we tried for `n_estimators`. A winner sitting on the edge of the grid means the optimum may lie outside it. Search again with higher values.

All fifteen combinations at once



✗ **Look where the winner sits: hard against the right edge of the grid. We did not find the optimum — we found the edge of what we searched.**

Preprocessing has hyperparameters too

The imputer lives inside the `ColumnTransformer` inside the `Pipeline`. The same double-underscore path reaches it, so it can be tuned by the same search:

```
grid = {"prep__num__simpleimputer__strategy":
            ["median", "mean", "most_frequent"]}
```

Imputation strategy	Cross-validated RMSE
`median`	$48,180
`mean`	$48,154
`most_frequent`	$48,171

A spread of $26 across the three. With 207 missing values out of 20,640, that is exactly what it should be — and now we know, instead of assuming.

The reason to put preprocessing in the pipeline is not that tuning it always helps. It is that preprocessing and model interact, and the pipeline is what lets you find out whether they do here.

Randomised search is usually better

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {"model__max_features": randint(low=2, high=13),
                  "model__n_estimators": randint(low=30, high=500)}

rnd_search = RandomizedSearchCV(full_pipeline, n_iter=10, cv=cv, n_jobs=-1,
                                param_distributions=param_distribs,
                                scoring="neg_root_mean_squared_error",
                                random_state=42)
```

- 1,000 iterations explores 1,000 *distinct values* of each hyperparameter; a grid explores only the few you listed
- Add a hyperparameter that turns out not to matter: a grid multiplies the runtime, a random search does not
- Six hyperparameters with ten values each is a million grid fits. Randomised search runs for however long you allow — and note that it cannot sit on the edge of a grid, because there is no grid

Two more ways to spend the budget

`HalvingGridSearchCV` and `HalvingRandomSearchCV` run a tournament: generate many candidates, evaluate them all on a small slice of the training set, promote the best with more data each round, and evaluate the finalists at full resource. Same search space, considerably less compute.

> **OR STOP TUNING AND COMBINE INSTEAD**
>
> Ensemble the models that perform best. A group will often beat its best individual member — just as a random forest beats the trees it is built from — but only when the members make **different kinds of errors**. Averaging correlated mistakes achieves nothing.

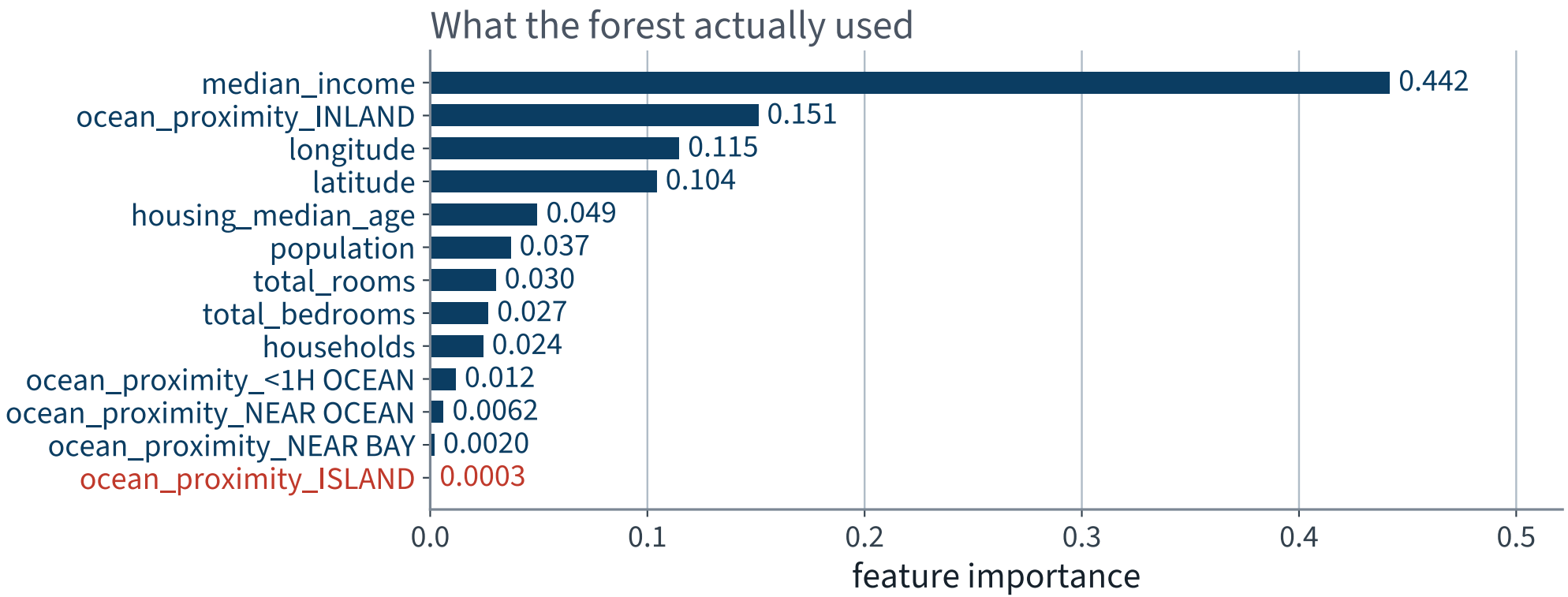
Lecture 7 makes “different kinds of errors” precise, by deriving the variance of an average of correlated predictors.

Analyse the best model

```
>>> sorted(zip(importances.round(4), feature_names), reverse=True)
[(0.4418, 'num__median_income'),
 (0.1511, 'cat__ocean_proximity_INLAND'),
 (0.1146, 'num__longitude'),
 (0.1043, 'num__latitude'),
 (0.0494, 'num__housing_median_age'),
 [...]
 (0.0062, 'cat__ocean_proximity_NEAR OCEAN'),
 (0.0020, 'cat__ocean_proximity_NEAR BAY'),
 (0.0003, 'cat__ocean_proximity_ISLAND')]
```

Median income dominates, as the correlation column predicted. Longitude and latitude together take 0.22 — the forest has rediscovered the coast from two raw coordinates. Of the ocean-proximity categories, only `INLAND` scores at all — though see the caveat on the next slide before concluding the others are worthless.

What the forest actually leaned on



Some of the one-hot ocean-proximity columns earn almost nothing. Importance is a reason to *try* dropping a feature, and then to measure — never a reason to drop it outright. We measure at the end of this lecture.

The gun we loaded in the previous lecture

Bottom of that list: `ocean_proximity_ISLAND`, at 0.0003. Five districts in California. Recall where they went:

ISLAND districts	Total	In training	In test
Stratified 80/20 on *income*, not on this	5	✗ 2	3

The model has seen the category twice. It will be asked about it three times, and we will report those three answers inside an average of 4,128.

AND IN CROSS-VALIDATION IT CAN VANISH ENTIRELY

`handle_unknown="ignore"` does not raise on a category it never saw. It emits `[0, 0, 0, 0]` — a district that is nowhere near the ocean and nowhere inland — silently.

With our seed no fold loses it; over 500 reshufflings, **11.2%** produce at least one fold in which the encoder is fitted without ISLAND and then asked about it.

A one-in-nine chance of a silently wrong column, invisible in the score. This is what “rare categories will cause trouble” meant.

Also check fairness

A model should work well not only on average, but across categories of district — rural or urban, rich or poor, northern or southern.

> Build subsets per category and analyse performance on each. If the model performs poorly on a whole category, it should not be deployed for that category — it may do more harm than good.

✓ We do exactly this in the last fifteen minutes of today, on the test set, and it changes what we would tell the client.

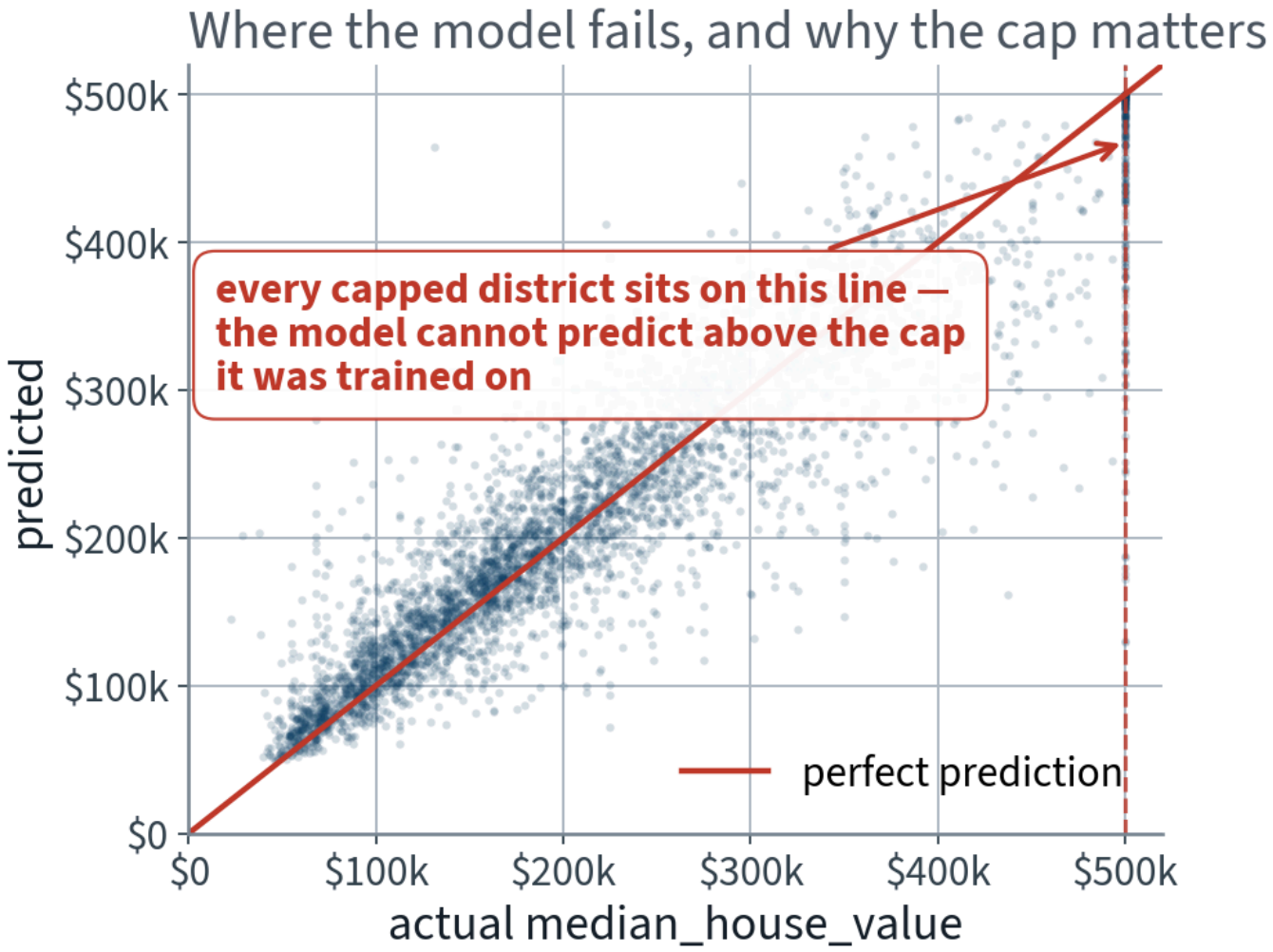
Finally: the test set

```
final_model = grid_search.best_estimator_    # refitted on all of X_train

X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)
```

$49,037

Note we `predict` on the test set. We never `fit` anything on it.

Predicted against actual, on data never seen



✗ **The stripe at $500,001 is the cap, coming back to collect: every district on it is one the model *cannot* get right.**

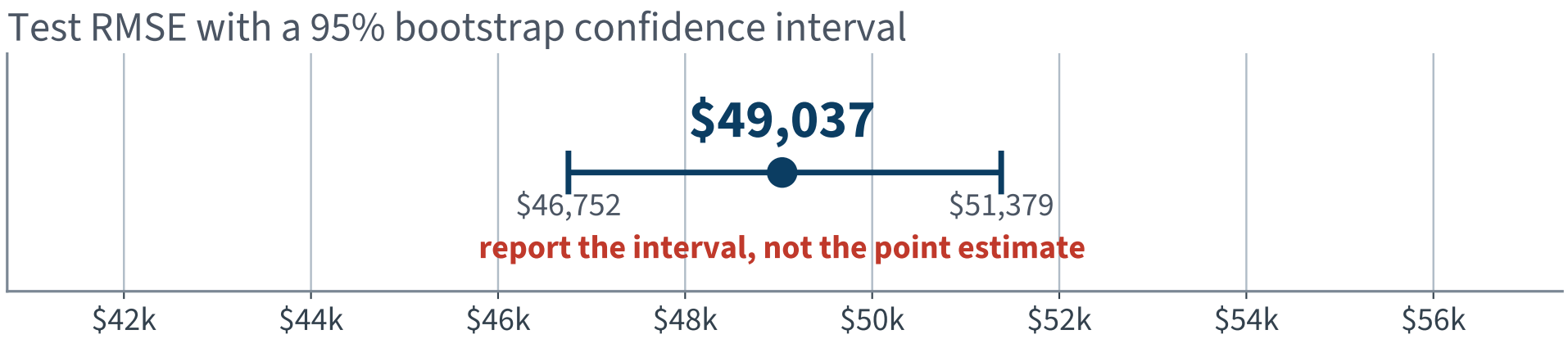
How precise is that estimate?

```
from scipy import stats

def rmse(squared_errors):
    return np.sqrt(np.mean(squared_errors))

squared_errors = (final_predictions - y_test) ** 2
boot_result = stats.bootstrap([squared_errors], rmse, confidence_level=0.95,
                              method="percentile",  # scipy defaults to BCa
                              random_state=42)
rmse_lower, rmse_upper = boot_result.confidence_interval
```

95% interval: **$46,752 – $51,379**.

> **WHY BOOTSTRAP AND NOT A T-INTERVAL**
>
> Not the skew — it is 8.9, but with $n = 4{,}128$ the CLT covers the *mean* anyway. It is that we report the mean's **square root**, and converting an interval for one into an interval for the other needs the delta method. The bootstrap skips it.

The number you should actually report



NOW USE IT ON THE OBVIOUS TEMPTATION

Tuned CV RMSE $48,180, test RMSE $49,037: $858 apart, against an interval of ±$2,313. **They agree. There is nothing here to explain.**

One last temptation

RESIST IT

If you tuned a lot, test performance is often slightly *worse* than cross-validated performance, because the system was fine-tuned to the validation data. When that happens you must **not** tweak hyperparameters to make the test number look better.

Those improvements would not generalise. You would simply be overfitting the test set instead — and the test set is the only thing standing between you and a confident fiction.

We searched 15 configurations, so the effect here is small enough to be invisible. Search 10,000 and it will not be.

Why tuning against the test set makes the test score optimistic, and why the bias is a statement about repeated splits rather than about this one.

EXAMINABLE

Look at the ten worst predictions

Actual	Predicted	Error	income_cat	ocean_proximity
$500,001	$129,715	✗ −$370,286	2	<1H OCEAN
$500,001	$130,590	✗ −$369,412	3	INLAND
$131,300	$464,117	✗ +$332,817	5	<1H OCEAN
$500,001	$171,694	✗ −$328,307	2	<1H OCEAN
$500,001	$175,534	✗ −$324,467	1	INLAND

Seven of the ten are capped districts the model under-predicts by a quarter of a million dollars. The third row is the mirror image: a district whose `median_income` is 15.0001 — the *other* cap — and which the model therefore prices as if it were Beverly Hills. It is worth $131,300.

✓ Both caps, spotted in a histogram before any model existed, are now the two largest errors of the finished system. Look at your worst cases. Always.

The average hides the model

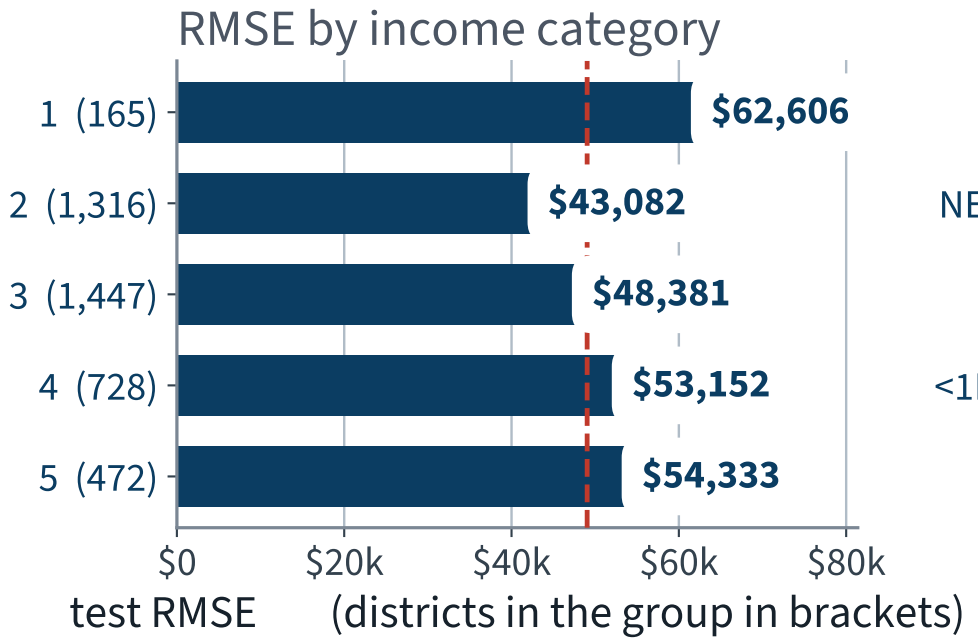
One number for 4,128 districts is a summary, not a finding. Break it out by the categories the client cares about:

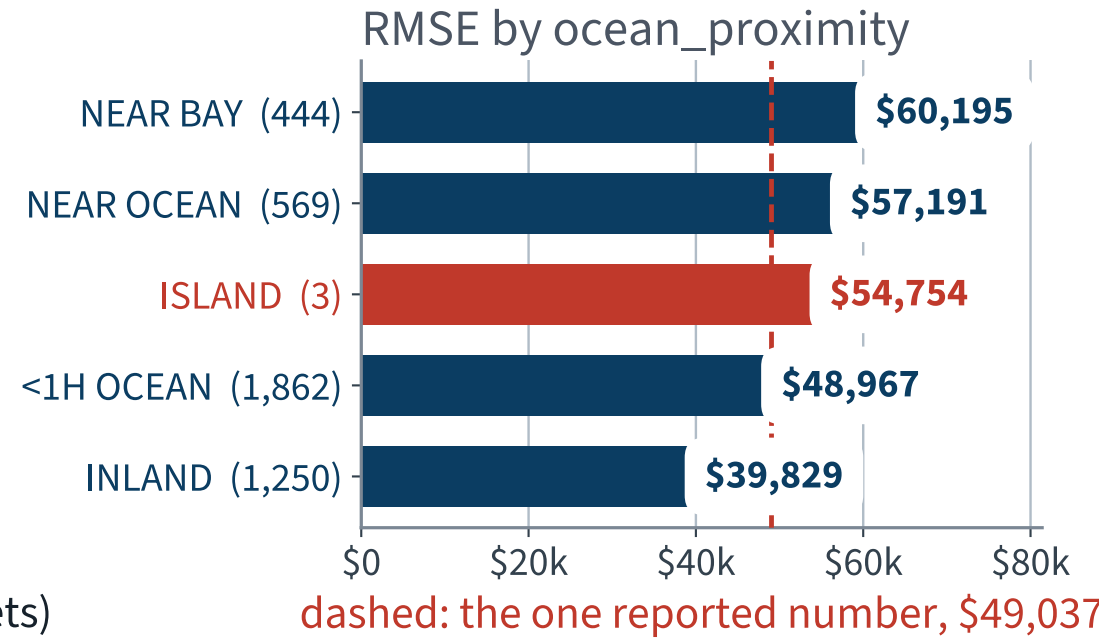
income_cat	n	RMSE
1 (poorest)	165	✗ $62,606
2	1,316	$43,082
3	1,447	$48,381
4	728	$53,152
5 (richest)	472	$54,333

ocean_proximity	n	RMSE
INLAND	1,250	$39,829 ✓
<1H OCEAN	1,862	$48,967
ISLAND	✗ 3	$54,754
NEAR OCEAN	569	$57,191
NEAR BAY	444	✗ $60,195

The poorest 165 districts are predicted worst in dollars — and their median price is $91,300, so in *relative* terms it is far worse still. NEAR BAY is 51% worse than INLAND. And the ISLAND number is computed from three districts: report it with the count or do not report it.

The same two slices, drawn





✗ A single RMSE of $49,037 describes none of these five groups. That is the answer to "also check fairness", and it is a different report to the client.

The obvious next move, and its trap

The cap causes the worst errors. So remove the 205 capped districts from the test set and measure again:

Evaluated on	n	Model RMSE	Constant baseline
All test districts	4,128	$49,037	$115,727
Capped districts removed	3,923	**$44,197 ✓**	$97,908

THESE TWO NUMBERS CANNOT BE COMPARED

$44,197 is not an improvement on $49,037. We did not change the model — we changed the **question**. The second row answers "how well does it do on districts below the cap?", which is a different and easier question. Notice that the baseline fell by 15% as well.

✗ Any time a number improves, ask first whether the evaluation set moved. Dropping rows is the cheapest way in the world to improve a metric, and it is almost never a result.

Should we drop them? For *training*, arguably yes — they teach the model a price that does not exist. For *reporting*, only with the sentence above attached.

The criterion we never measured

The brief said the experts are off by more than **30% of the price**. We optimised dollars. Here is what that cost — measured on the **training folds**, because comparing two differently-trained models is a *choice*, and choices are not made on the test set:

Trained on	RMSE	Median error, as % of price	Within 30%
the price (what we did)	**$48,629 ✓**	12.0%	85.0%
log of the price	$49,646	**11.5% ✓**	**86.5% ✓**

Exactly the trade the metric slide predicted: regressing the log target is $1,017 worse in dollars and **1.5 points** better on the criterion the stakeholder actually stated. Neither is *the* right answer — but choosing without measuring is not a choice.

> **WHERE THIS COMPARISON IS ALLOWED TO HAPPEN**
>
> Five-fold, on the training rows — doing it on the test set is the procedure this deck's own specimen exam answer marks **wrong**. The test pair reads $49,037 / $49,955 and chose nothing.

THE LAST FIFTEEN MINUTES

What the number means

15 minutes

The three numbers that matter

Measured how	RMSE	vs the baseline
Predict a constant — the training mean	$115,311	—
Best model, cross-validated on the training set	$48,180	58% better
The same model, on the test set, once	**$49,037 ✓**	58% better

READ THE THIRD ROW AGAINST THE SECOND

$49,037 is $858 worse than the cross-validated estimate, against a 95% interval of ±$2,313. The estimate was honest, and there is nothing in that $858 to explain.

The first row is the one people leave out. Without it, $49,037 is a number with no units of judgement attached to it.

Is that good?

On the stakeholder's own criterion, the system is within 30% on **85.1%** of districts, and its median miss is **11.3%**. The experts were said to be off by more than 30% typically. So: plausibly better — on an assumption we were handed and never checked.

> **WHAT A DEFENSIBLE REPORT SAYS**
>
> Ask for fifty districts estimated by the expert team, and score them on the same test rows with the same metric. Until then the comparison is between a measurement and a claim, and only one of them has an interval.

It may still be worth launching — especially if it frees the experts for work only they can do. But do not deploy it unqualified for the poorest districts, or for the 205 at the cap.

"Is the model good?" is not a question about RMSE. It is a question about what the organisation does with it.

Six questions to ask of any reported number

- Was every transformer fitted *after* the split, on training data only?
- Is preprocessing inside the pipeline passed to cross-validation?
- Was the reported metric computed on held-out data?
- Was the test set touched more than once?
- Were hyperparameters selected using validation data, not test data?
- Does any feature encode information unavailable at prediction time?

Keep these. They catch most of what goes wrong in this course and in your own projects, and questions 1 and 2 are what Part B of the paper asks when it hands you a code excerpt.

Can an assistant check this for you?

Partly — and only if you ask it as an *adversary*, not as an author. This is step 3 of the loop from Lecture 1, delegated.

> “Is this code correct?”
>
> Invites agreement. You will usually get it.

> “This notebook reports RMSE $49,037. Assume that number is optimistic. List every path by which test-set information could have reached the training procedure, and cite the line.”

Give it the failure to look for, and a number to disbelieve. Generic review requests return generic praise.

What that catches — and what it does not

Bug	Found from the code alone?
`fit_transform` on the full set	**yes ✓**
Preprocessing outside the cross-validated pipeline	**yes ✓**
Test set evaluated more than once	**no — that is session history, not code ✗**
A feature unavailable at prediction time	**no — that is domain knowledge ✗**
Split not stratified on the right variable	**no — "right" is your judgement ✗**

✗ Three of the five require something the assistant cannot read off the file. Those three are yours.

Moving the check upstream

Every failure named today is preventable at specification time. Keep this as a standing constraint in every request:

> **STANDING CONSTRAINT**
>
> “Split before anything is fitted. All preprocessing lives inside a `Pipeline` passed to cross-validation. Nothing derived from the test set may appear in the training path. Fixed random seed. Print the fold scores, not just the mean.”

A specification is cheaper than a diagnosis. Every notebook in this course is written against it, which is why none of them contains the failures above.

Back to the specimen question — 3, the repair

The specimen question from the last lecture: a loop choosing α by scoring on `X_test`. You could answer 1 and 2 then. You can answer 3 and 4 now, because cross-validation exists.

```
gs = GridSearchCV(Ridge(), {"alpha": [0.01, 0.1, 1, 10, 100]},
                  scoring="neg_root_mean_squared_error",
                  cv=KFold(10, shuffle=True, random_state=42)
                  ).fit(X_train, y_train)

print(root_mean_squared_error(y_test, gs.predict(X_test)))
```

WHAT MAKES IT DEFENSIBLE

α is chosen on validation folds carved out of the *training* set. The test set is touched **exactly once**, after α is already fixed. It is the same two-line shape we used on the forest an hour ago.

`RidgeCV` does the same job in one line and is equally acceptable. What is *not* acceptable is any version in which `X_test` appears before the choice is made.

Specimen answer — 4

In expectation?

✓ Higher.

- The printed number was a *minimum* over five noisy estimates, so it sits below the truth
- α was chosen to look good on the very set it was scored on

Guaranteed on one split?

✗ No.

- The bias is a statement about *repeated draws*, not about this sample
- Cross-validation may also pick a genuinely better α

✗ Both halves are required. *Higher* alone, or *no* alone, is half the marks.

You have now seen the second half measured: our own tuned model came out $858 higher on the test set, which is well inside the interval — a single split, telling you nothing about the expectation.

Presenting it, and making it reproducible

```
joblib.dump(final_model, "my_california_housing_model.pkl")   # with its scores
```

The report

- Concise; visualise the key insights
- Technical for peers, high-level for stakeholders
- One memorable statement — here: *median income is the number one predictor, and the model should not be trusted at the cap*
- What worked, what did not, what you assumed, where it is limited

The artefact

- Code the team can run, with a structured README
- Notebooks where code, explanation and results sit together
- `requirements.txt` or a container image, pinning versions
- Seeds for every generator; `KFold(shuffle=True, random_state=42)`, never the default

✗ A result nobody else can reproduce is not a result — and at the oral, “it worked when I ran it” is not an answer.

Save every model you experiment with together with its cross-validation scores and its validation predictions, so you can compare the *kinds* of errors they make. And deployment is not the end: performance decays quietly through *data drift*, so it has to be monitored.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

What we did

1. Derived the normal equation: fitting a linear model is orthogonal projection, and $\mathbf{X}^\mathsf{T}\mathbf{X}$ is invertible exactly when the columns are independent
2. Saw why it fails — collinearity, or $m < n$ — and that the SVD pseudoinverse does not
3. Established that a score computed on the training rows cannot detect overfitting, and that the test set is spent the first time it influences a choice
4. Named data leakage as a condition, not an accident, and built the `Pipeline` that makes it structurally impossible
5. Used k-fold cross-validation for an honest estimate *and* its precision — two estimates differ only if the paired per-fold difference says so
6. Tuned with grid and randomised search, then touched the test set once and reported an interval
7. Sliced the error: an average over 4,128 districts describes none of them, and two numbers measured on different rows are not comparable

In the book

Topic	Chapter
Normal equation, SVD, pseudoinverse, complexity	4 · §4.1, brought forward
Cross-validation, grid and randomised search	2
Pipelines and `ColumnTransformer`	2
Overfitting, generalisation error, validation sets	1, 2
Feature importance, error analysis, deployment	2
Transforming the target; `TransformedTargetRegressor`	2
Nonparametric models and tree regularisation	5

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 2

five, in the style of the written paper

Exercises — Lecture 2

1 The normal equation is $\hat{\theta} = (X^\mathsf{T} X)^{-1} X^\mathsf{T} y$. Your design matrix has a column equal to the sum of two others. State what happens to $X^\mathsf{T} X$, and what `np.linalg.pinv` returns instead. [5]

2 A pipeline scales the features and then fits a ridge model, and is passed whole to `cross_val_score`. Explain what would go wrong if the scaling were done once, before the call. [5]

3 You report a 95% confidence interval for the test RMSE. A colleague says the interval proves the model is better than the baseline. Give the one question you would ask before agreeing. [4]

Solutions on Lecture 3's deck.

Exercises — Lecture 2 (continued)

4 k-fold cross-validation reports a mean of 0.71 with folds $[0.52, 0.79, 0.75, 0.74, 0.75]$. What do you conclude, and what would you do next? [5]

5 Explain, in two sentences, why the test set may be looked at exactly once, and what you may legitimately do if the number disappoints. [4]

Solutions on Lecture 3's deck.

THE FIVE SET AT THE END OF LECTURE 1

Solutions Lecture 1

not part of this lecture

Solutions — Lecture 1

1. A colleague reports that their model reaches an RMSE of $48{,}000$ on the California housing data, and that…

✓ It estimates the best-of-six score on that particular split; it does not estimate the error on new data.

- Choosing the minimum of six numbers measured on the same data makes that minimum a *selection*, not a measurement
- The quantity a client cares about — error on districts nobody has seen — needs data that took part in no choice

2. The median income column is capped at 15. Give one consequence for the model and one for the evaluation, and…

✓ Model: it can never learn what happens above the cap. Evaluation: the test set is capped too, so the error is silent about exactly those districts.

- The cap is in both halves of the split, so no held-out number reveals it
- Raise the evaluation one first: the model can be retrained, but a metric that cannot see the failure will not tell you to

Solutions — Lecture 1 (2)

3. You are given a stratified split on income category and a naive random split of the same data. Both report…

✓ No.

- Similar RMSE on *one* draw says nothing about the spread over draws, and the stratification is there to reduce that spread
- The failure a naive split causes is occasional and large, which is precisely what a single comparison cannot show

4. The brief says the output feeds a downstream system that expects a price. Name the one thing this fact…

✓ It settles that this is supervised regression, not classification. It leaves the metric open.

- 'A price' fixes the type of the output and nothing about how error is weighed
- Whether being wrong by $50{,}000$ on a cheap district matters as much as on an expensive one is a question for the client, not the data

Solutions — Lecture 1 (3)

5. In the working loop — specify, generate, read, test, verify — exactly one step is done by the…

✓ Generate. If reading is also delegated, nothing checks that the code does what the specification asked for.

- The specification is the student's claim about what the code must do
- Only a reader who holds that claim can notice the code meeting a different one

LECTURE 3 · GÉRON CH. 3

Classification and its metrics

Why accuracy is useless under imbalance, and why precision is not monotone in the threshold
The confusion matrix, precision, recall, F1
PR and ROC curves, and choosing an operating point you can defend

Run the Lecture 2 notebook first